

CLASS & OBJECT PRACTICE SET

1. Creating a Simple Class and Objects

Question:

Create a class `Animal` using `pass` and make two objects.

Code:

```
class Animal:
    pass

dog = Animal()
cat = Animal()

print(type(dog))
print(type(cat))
```

Try Yourself:

Create a class `Car` with no attributes. Make three objects: `c1` , `c2` , and `c3` .

2. Constructor and Instance Attributes

Question:

Create a class `Person` with attributes `name` and `age` .

Code:

```
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age

p1 = Person("Ram", 32)
p2 = Person("Hari", 23)

print(p1.name, p1.age)
print(p2.name, p2.age)
```

Try Yourself:

Create a class `Book` with `title` and `author` . Create two books and print their details.

3. Instance Methods

Question:

Add a method to display student info.

Code:

```
class Student:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def show_info(self):
        print("Name:", self.name)
        print("Age:", self.age)

s1 = Student("Ram", 12)
s1.show_info()
```

Try Yourself:

Create a class `Mobile` with `brand` and `price` . Add a method `mobile_info()` .

4. Understanding the self Keyword

Question:

Show that `self` refers to different objects.

Code:

```
class Student:
    def __init__(self, name):
        self.name = name

    def greet(self):
        print("Hello, my name is", self.name)

s1 = Student("Ram")
s2 = Student("Hari")

s1.greet()
s2.greet()
```

Try Yourself:

Create a class `Player` with `name` and method `introduce()` .

5. Class Attribute vs Instance Attribute

Question:

Demonstrate a class attribute.

Code:

```
class Student:
    school = "ABC School"
```

```
def __init__(self, name, age):
    self.name = name
    self.age = age

s1 = Student("Ram", 12)
s2 = Student("Hari", 14)

print(Student.school)
print(s1.school)
print(s2.school)
```

Try Yourself:

Create a class `Laptop` with class attribute `company = "Dell"` .
Create two laptops with different model names and prices.

6. Class Method

Question:

Print a class attribute using a class method.

Code:

```
class Student:
    school = "ABC School"

    @classmethod
    def show_school(cls):
        print("School Name:", cls.school)

Student.show_school()
```

Try Yourself:

Create a class `Country` with class attribute `continent = "Asia"` and class method `show_continent()` .

7. Creating Multiple Objects and Looping

Question:

Store multiple students in a list.

Code:

```
class Student:
    def __init__(self, name, age):
        self.name = name
        self.age = age

students = [
    Student("Ram", 12),
    Student("Hari", 14),
    Student("Sita", 13)
]

for s in students:
    print(s.name, s.age)
```

Try Yourself:

Create three `Employee` objects with `name` and `salary`. Loop through and print their info.

8. Multiple Methods Inside a Class

Question:

Create methods to calculate the area and perimeter of a rectangle.

Code:

```
class Rectangle:
    def __init__(self, length, width):
        self.length = length
```

```
        self.width = width

    def area(self):
        return self.length * self.width

    def perimeter(self):
        return 2 * (self.length + self.width)

r1 = Rectangle(5, 3)
print("Area:", r1.area())
print("Perimeter:", r1.perimeter())
```

Try Yourself:

Create a class `Square` with methods `area()` and `perimeter()` .

9. Updating Instance Attributes

Question:

Add a method to modify age.

Code:

```
class Player:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def update_age(self, new_age):
        self.age = new_age

p1 = Player("Ravi", 20)
p1.update_age(25)
print(p1.age)
```

Try Yourself:

Create a class `Employee` with method `update_salary(new_salary)` .

10. Using an Object Inside a Function

Question:

Write a function that accepts a Student object.

Code:

```
class Student:
    def __init__(self, name, age):
        self.name = name
        self.age = age

def print_student(s):
    print("Name:", s.name)
    print("Age:", s.age)

s1 = Student("Ram", 12)
print_student(s1)
```

Try Yourself:

Create a class `Book` and a function `show_book(b)` that prints details.

11. Default Constructor Values

Question:

Give a default age if not provided.

Code:

```
class Person:
    def __init__(self, name, age=18):
        self.name = name
        self.age = age
```

```
p1 = Person("Sam")
p2 = Person("Hari", 25)

print(p1.name, p1.age)
print(p2.name, p2.age)
```

Try Yourself:

Create a class `Course` with default duration = "3 months".

12. Counting Objects (Class Attribute)

Question:

Count how many Student objects were created.

Code:

```
class Student:
    count = 0

    def __init__(self, name):
        self.name = name
        Student.count += 1

s1 = Student("Ram")
s2 = Student("Hari")
s3 = Student("Sita")

print("Total students:", Student.count)
```

Try Yourself:

Create a class `Order` that counts how many orders are created.

13. Storing Lists Inside a Class

Question:

Store a list of players inside a team.

Code:

```
class Team:
    def __init__(self, name):
        self.name = name
        self.players = []

    def add_player(self, player_name):
        self.players.append(player_name)

t1 = Team("Warriors")
t1.add_player("Ram")
t1.add_player("Hari")

print(t1.players)
```

Try Yourself:

Create a class `Library` with a list of book names. Add method `add_book()` .

14. Returning Formatted Information

Question:

Return formatted text instead of printing.

Code:

```
class Student:
    def __init__(self, name, grade):
        self.name = name
        self.grade = grade

    def get_info(self):
```

```
        return f"Name: {self.name}, Grade: {self.grade}"

s1 = Student("Ram", "A")
print(s1.get_info())
```

Try Yourself:

Create a class `Product` with `name` and `price`. Add a method `details()`.

15. Difference Between Class and Object Attributes

Question:

Show that modifying an instance attribute does not affect the class attribute.

Code:

```
class Laptop:
    company = "Dell"

    def __init__(self, model):
        self.model = model

l1 = Laptop("Inspiron")
l2 = Laptop("Vostro")

l1.company = "HP"

print(l1.company)
print(l2.company)
print(Laptop.company)
```

Try Yourself:

Create a class `School` with class attribute `city = "Kathmandu"`.

Change city for only one object and compare results.
